

Scenario Resolver

A guided reading of `resolve_scenario.py`

Master's Thesis — *Asset-Centric Temporal Graphs for Crude-Oil Logistics: Schema Design and Consistency Guarantees*

Pedro Porfirio · NOVA IMS · Universidade Nova de Lisboa

Supervisor: Professor Flavio Pinheiro

Document: **Resolver Walkthrough** · v1.0 · 2026-05-14

1. Why the resolver exists

Before the resolver, every consumer that wanted to read a variable's value had to re-implement resolution logic itself. The balance UI carried a 100-line CTE handling TS bindings, mirror formulas, reverse-paired-edge inheritance, and closure-equation evaluation. JavaScript walked arithmetic formulas at render time, because SQL could not evaluate multi-term expressions of the form $A - B - C$. Each new generator (hierarchy explorer, geographic map, Chapter-5 validation queries, future forecasting features) would have duplicated that work.

The resolver collapses every consumer's resolution logic into a single deterministic pass:

```
inputs    : variables + variable_assignments + timeseries_data
           + the active scenario

output    : one row per (variable, date) in scenario_resolved_values
```

Every downstream consumer becomes a thin `SELECT` over the result table. The resolver runs in roughly twenty seconds for the starter scenario (1,830 variables $\times$ 156 monthly observations); it is run once per assignment change, not once per consumer.

2. The output table

The resolver writes to `oil_network.scenario_resolved_values`. Two columns from the DDL deserve particular attention:

```
CREATE TABLE oil_network.scenario_resolved_values (
  scenario_id      TEXT NOT NULL REFERENCES scenarios      ON DELETE CASCADE,
  variable_id      TEXT NOT NULL REFERENCES variables      ON DELETE CASCADE,
  observation_date  DATE NOT NULL,
  value            DOUBLE PRECISION,
  source           TEXT NOT NULL CHECK (source IN
    ('observed', 'derived', 'zero', 'latent', 'unresolved')),
  formula_used     TEXT,
  timeseries_id    TEXT,
  saved_date       TIMESTAMPTZ DEFAULT NOW(),
  run_id           BIGINT REFERENCES scenario_resolver_runs,
  PRIMARY KEY (scenario_id, variable_id, observation_date)
);
```

The value column is nullable. A NULL value paired with `source = 'latent'` is a valid and intentional row — it tells downstream consumers “this variable is declared but its value is unknown by design; do not retry.” This is the schema-level expression of Principle 2.11 (latent allocation at junctions): the resolver records that a flow exists structurally and is constrained by mass balance, but its per-edge value is not pinned down by observation.

The source column is a CHECK-constrained enum with five categories:

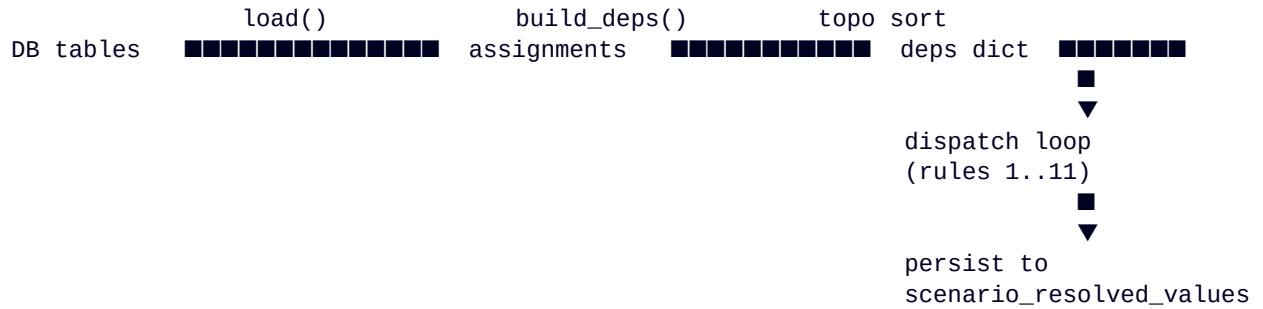
source	meaning
observed	Direct time-series lookup — ground-truth measurement.
derived	Value computed by a formula (alias, sum, closure, arithmetic, mirror).
zero	Structural zero (e.g. a refinery does not produce crude).
latent	Declared variable with no resolution path (value = NULL by design).
unresolved	A formula was given but could not be evaluated. Zero in a healthy run.

`formula_used` preserves which resolution rule fired (the dispatch labels in Section 4). This is for auditing: it lets a reviewer reconstruct, for any value, the exact path that produced it.

Every run also writes an audit row to `scenario_resolver_runs` with a started/completed timestamp, duration, dispatch counts as JSONB, and optional free-text notes. Every resolved-value row carries the `run_id` that produced it, so historical comparison is a single SQL join.

3. How a run flows, end to end

The top-level shape of `resolve()` follows five steps:



Each phase has one purpose, isolated from the others. The dispatch loop never reads from the DB; the loader never evaluates formulas; the topological sort never produces values. Mistakes in one phase are diagnosed by inspecting its single concern.

4. The dispatch loop: ten resolution rules

Rules are tried in priority order. The first rule that succeeds produces a value and short-circuits the rest. Rule 11 (unresolved) is a fallback that should fire zero times in a healthy run.

The full set of canonical rule labels — that is, the only `source` values that `scenario_resolved_values` ever records — is short. Three earlier labels were collapsed into the canonical `sum` in the twelfth pass; they no longer appear in the codebase or the resolved table:

Canonical label	Retired predecessors
observed	—
zero	—
latent	—
sum	sum_over_children, sum_over_outflows, sum_same_type — semantic role recoverable from formula_inputs structure, see Design_Principles § The canonical sum rule
alias	—
reverse_mirror	—
arithmetic	—
closure	dormant in starter scenario after B promoted to TS-observed (sixth pass)
unresolved	fallback; zero rows expected

The full per-rule details follow.

#	Rule	Description
1	Observed	<code>timeseries_id</code> IS NOT NULL. Direct TS lookup. This is the ground truth; every other rule is downstream of these. In the starter scenario, 80 variables fire this rule.
2	Structural zero	<code>formula = '0'</code> . Used for refinery production, pass-through inventory, and other slots where the value is zero by physical construction. Roughly 628 variables fire this rule, mostly non-relational scalars inherited from <code>node_type_default_formulas</code> .
3	Latent, with reverse-mirror promotion	<code>formula = 'latent()'</code> . Records value = NULL and source = 'latent'. But first, the rule attempts a <i>reverse-mirror promotion</i> : if the paired direction (opposite <code>variable_type</code> , swapped endpoints) has a resolved value, the rule borrows it via the mirror identity. Without this promotion, a latent <code>outflow_to_padd2</code> on <code>canadian_oil_sands</code> would beat the reverse-mirror rule and leave the variable NULL despite a known value on the inflow side of the same edge.

#	Rule	Description
4	Sum	<code>formula = 'sum'</code> . Sums every value in <code>formula_inputs</code> ; NULL inputs are skipped. This is the twelfth-pass collapse of three earlier labels (<code>sum_over_children</code> , <code>sum_over_outflows</code> , <code>sum_same_type</code>) into one canonical rule. The semantic role — aggregation parent, fan-out total, residual — is recoverable from the structure of <code>formula_inputs</code> (same-type and same-commodity $\Rightarrow$ aggregation; mixed-type at same node $\Rightarrow$ fan-out), so the formula text does not need to encode it.
5	Single-variable alias	<code>formula = bare_variable_id</code> and the <code>formula_inputs</code> list contains at most that one entry. The variable inherits its value from the named target. Most aggregate-to-aggregate alias chains fire this rule; in the starter scenario, 442 variables resolve via alias.
6	Reverse mirror	Relational variable with no own resolution; its paired direction (opposite type, swapped endpoints) has a resolved value. The variable borrows that value. After the ninth-pass fix, this rule fires deterministically — <code>build_deps()</code> now guarantees the paired side is resolved before this side is evaluated, so 15 mirrors fire reliably (previously 3 by luck of insertion order).
7	Closure formula	Triggers when the variable is a <code>balancing_item</code> whose <code>formula_inputs</code> include a mix of inventory, inflow, and outflow variables. Evaluates $B = \Delta S - P + C - \Sigma I + \Sigma O$. Now dormant in the starter scenario because B was promoted to TS-observed in the sixth pass (MCRUA series).
8	Arithmetic combination	Parses the formula text as signed variable references (<code>A - B + C</code>). Used for residual definitions: <code>padd2_other.P = padd2_view.P - bakken_nd.P - oklahoma.P</code> .
9	Unresolved	Records <code>value = NULL</code> , <code>source = 'unresolved'</code> . The fallback. In a healthy run this fires zero times; non-zero counts indicate a bug or an unmodelled formula pattern.

5. Building the dependency DAG

`build_deps()` walks every assignment and produces a dict `deps[variable] = set of upstream variable_ids`. The set must be acyclic so that `graphlib.TopologicalSorter` can produce a valid evaluation order. The build has three sources of dependencies:

(a) formula_inputs. Every variable id listed in `formula_inputs` that exists in the assignment set becomes a dependency.

(b) sum_over_outflows. The variable depends on every outflow variable at the same node, except itself.

(c) Reverse-mirror dep, hoisted above the early continue. When a variable is latent and its paired direction is resolved, the paired direction is added as a dependency. Two cycle-avoidance gates protect this: (i) the paired side must not itself be plain latent, else neither side has a value; and (ii) the paired side must not already alias from us, else we get a $us \leftrightarrow$ paired loop. The second gate is the `paired_aliases_us` check added in the ninth pass.

Cycle gates matter because the topological sort throws `CycleError` on the first cycle. The resolver tolerates many alias and mirror patterns precisely because of these gates; the cost is a handful of subtle conditions in `build_deps()` that need to be read together.

6. The closure formula

Rule 8 evaluates the mass-balance closure when the parent variable is a `balancing_item` whose inputs span the requisite variable types:

$$B = \Delta S - P + C - \Sigma I + \Sigma O$$

$$\Delta S_{\text{kbd}} = (S(t) - S(t-1)) / \text{days_in_month}(t) \quad \# \text{ kbd, not MBBL}$$

If any required input is NULL at date `t`, the closure is skipped for that date – the resolver does not silently substitute zero.

Skipping rather than zero-imputing matters: a NULL inflow with a non-zero `P` and `C` would produce a spurious `B` if treated as zero, and that `B` would propagate into the USA aggregate. Skipping preserves the signal that some upstream value is genuinely missing.

Since the sixth pass, `B` at PADD and USA level is TS-observed (MCRUA series), so closure is no longer the primary path. It remains as a fallback constraint — particularly valuable for the Chapter-5 Claim-4 view, which compares closure-derived `B` against observed `B` and reads the delta as the magnitude of partition-internal latent flow.

7. Arithmetic formula evaluator

Rule 9 parses formula strings of the form $A - B - C$ via a small regex that pulls out signed terms. Each term must match a variable id present in `formula_inputs`; anything that fails to match aborts the rule cleanly and falls through to the next rule. This limits the surface for parser exploits and keeps the formula language human-readable.

```
RE_TERM = re.compile(r'([+-]?)\s*([a-z][a-z0-9_]*)')

for m in RE_TERM.finditer(formula):
    sign = -1 if m.group(1) == '-' else 1
    tok = m.group(2)
    if tok not in input_set:
        ok = False; break
    terms.append((sign, tok))
```

Eight variables in the starter scenario fire this rule. They are structural residual definitions such as `padd2_other.P = padd2_view.P - bakken_nd.P - oklahoma.P`, where the residual exists precisely to soak up the difference between an aggregate total and its enumerated constituents.

8. Persistence and the audit trail

After the dispatch loop finishes, the resolver clears the scenario's existing rows in `scenario_resolved_values` and writes the new ones via `execute_values` in pages of 5,000. The audit row in `scenario_resolver_runs` is updated with the completion timestamp, the elapsed milliseconds, and the dispatch counts as JSONB.

If the resolver crashes mid-way, the open audit row with NULL `completed_at` is itself a useful signal — it tells the next operator that a run was started but did not finish. The resolved-values table is unaffected by a crash because the DELETE/INSERT pair runs inside a transaction.

9. Bugs encountered and fixed

9.1 The latent-mirror cycle

Original implementation: a latent variable would add its paired direction as a dependency unconditionally. When the paired direction was also latent and aliased to its own paired side, this produced a two-node cycle ($us \rightarrow \text{paired} \rightarrow us$) and `TopologicalSorter` raised `CycleError`. Fix: require the paired side to be non-latent before adding the dep (first cycle gate).

9.2 The latent short-circuit that hid reverse-mirror

Rule 3 (latent) ran before Rule 7 (reverse-mirror), so a variable explicitly marked latent would record `value = NULL` even when its paired direction had an observed value. Fix: promote the mirror inside Rule 3 — if the paired side is resolved, borrow its value first and only fall through to `NULL` otherwise. Together with the `build_deps` hoist (so that latent variables get their paired-side dep added before being skipped), this lifted mirror dispatch from 3 (by luck of insertion order) to 15 (deterministic).

9.3 The alias-rule wrapper bug

Rule 6 matches when `formula in by_id`, i.e. the formula text is itself a bare variable id. Earlier migration scripts wrote `formula = 'alias(variable_id)'` with the wrapper string; rule 6 missed those, and they slipped past on Rule 10 (same-type rollup) by coincidence. The fix is in the migration scripts: `formula = bare_variable_id`, with the wrapper retained only as the display label in `formula_used` on resolved rows.

10. Dispatch counts in the current starter scenario

Run id = 14, dated 2026-05-14 (twelfth pass, after the sum-rule collapse and JSONB cleanup). 1,830 assignments, 156 monthly dates, 223,114 resolved rows persisted in roughly seventeen seconds.

Rule	Count
observed (TS lookup)	80
zero (structural)	628
latent (incl. reverse-mirror failures)	652
alias (single-variable)	442
reverse-mirror (promotion + Rule 6)	15
arithmetic	8
sum (canonical — collapses sum_over_*/sum_same_type)	5
closure ($B = \Delta S - P + C - \Sigma I + \Sigma O$)	0
unresolved	0

Zero unresolved is the load-bearing invariant. Every declared variable has a value or an explicit latent marker. The downstream consumers can rely on the presence of a row for every (variable, date) pair, and on the source column telling them whether the NULL is by design (latent) or by failure (unresolved, which is an unmodelled case that needs attention).